FEWER EVENT HANDLERS

The `XDomainRequest` object only has four event handlers: `onload`, `onerror`, `onprogress`, and `ontimeout`. These behave the same as their `XMLHttpRequest` counterparts. There is no `onabort` method, because there is no corresponding `abort` method to abort the request, and there aren't `onloadstart` and `onloadend` methods to mark the beginning and end of the request.

`HMLHttpRequest`'s `onloadend` handler is useful for ensuring a piece of code always executes, regardless of whether the request was a success or a failure. For example, the `onloadend` handler could be used to hide a status message at the end of the request. If you need to execute a piece of code at the end of a request, be sure to put it in the `onload`, `onerror`, and `ontimeout` methods.

LESS RESPONSE INFORMATION

Once the server sends an HTTP response, the `XDomainRequest` object only gives the JavaScript code access to the response body and response content type. There is no way to access the HTTP status code or status text, or any of the response headers.

The `XMLHttpRequest` and `XDomainRequest` objects allow HTTP requests to be made from JavaScript. But these aren't the only way the browser uses CORS. The next section covers how the HTML5 `<canvas>` element uses CORS to load images.

2.5 Canvas and cross-origin images

HTTP requests from JavaScript are the most common way to use CORS, but they aren't the only way. The HTML `<canvas>` element also relies on CORS when loading images from different origins. Canvas was introduced in HTML5 as a way to draw shapes and images from JavaScript. The following listing shows how to use JavaScript and a canvas to draw a solid rectangle. Figure 2.7 shows what this canvas looks like in the browser.

Listing 2.8 Drawing a rectangle on a canvas

```
<canvas id="myCanvas"></canvas>
<script>
var myCanvas = document.getElementById('myCanvas');
var myContext = myCanvas.getContext('2d');
myContext.fillStyle = '#888';
myContext.fillRect(0,0,240,150);
</script>
```

In addition to drawing shapes, a canvas can also display images. These images can live either on the same server as the page, or on a different server. The canvas can display all images, regardless of whether or not the image comes from the same origin. The difference is that cross-origin images can taint the canvas.

Tainting a canvas means that data can no longer be extracted from the canvas. The `<canvas>` element exposes three methods for extracting data: `toBlob`, `toDataURL`, and `getImageData`. All three methods return the binary image data. When these methods are called on a canvas with a same-origin image, they work just fine. But when they are called on a canvas with a cross-origin image, the browser throws an error, as shown in figure 2.8.